BÁO CÁO CÁ NHÂN · BÀI TẬP LỚN DEVSECOPS

Kiểm soát luồng phân nhánh bằng Policy-as-Code cho hệ thống InlaneWeb

Ngăn mã nguồn chưa được rà soát đi thẳng lên production,
nơi lưu dữ liệu cá nhân của khách hàng

Sinh viên	**Nguyễn Giang Nam** · B26CHAT030
Vai trò	Trưởng nhóm · Kiến trúc sư DevSecOps
Nhóm · Lớp	Nhóm 11 · M26CQAT01-B
Học phần	DevOps và DevSecOps (SEC4342) · PTIT
Mã nguồn	github.com/wtfwhoyouare/PTIT-DevOps-SonarQube

BỐ CỤC

Nội dung trình bày bám theo 4 yêu cầu của đề bài

Yêu cầu	Nội dung	Slide
YC1	Mô tả ứng dụng InlaneWeb, dữ liệu cá nhân cần bảo vệ và 3 quy trình nghiệp vụ liên quan tới an toàn	3 – 5
YC2	Môi trường triển khai và đối chiếu với các yêu cầu an toàn tối thiểu	6 – 7
YC3	Vấn đề an toàn cá nhân phụ trách: cách phát hiện, cách xử lý, chính sách bị vi phạm (NĐ 13/2023, GDPR), pha DevSecOps, bài toán Policy-as-Code	8 – 12
YC4	Demo đoạn code phát hiện và chặn lỗi, kết quả chạy thật trên máy và trên GitHub Actions	13 – 15
	Điểm hở phát hiện thêm, cách đã khắc phục và kết luận	16 – 17

Phân loại trong nhóm: vấn đề của tôi thuộc nhóm lỗi **triển khai / vận hành**. Các thành viên khác phụ trách nhóm lỗi **mã nguồn** (SQL Injection, LFI, XSS), đáp ứng yêu cầu nhóm có tối thiểu 2 nhóm lỗi.

YC1 MÔ TẢ ỨNG DỤNG

InlaneWeb: cổng thông tin vận tải đường biển

Ứng dụng web PHP 8.2 + MariaDB 10.11 chạy trên Docker. Khách hàng tra cứu cảng biển, theo dõi vận đơn, bình luận trên đơn hàng; quản trị viên xem dashboard và xuất báo cáo.

- **Công khai:** trang chủ, tin tức, tra cứu cảng, hướng dẫn nghiệp vụ
- **Khách hàng:** đăng nhập, tài khoản, vận đơn, bình luận
- **Quản trị:** dashboard, console, xuất và tải báo cáo
- **Vận hành:** endpoint `/health` phục vụ smoke test và tự phục hồi

Đây là ứng dụng **cố ý chứa lỗ hổng** (vulnerable-by-design) để nhóm thực hành phát hiện và khắc phục.

Dữ liệu cá nhân cần bảo vệ

Bảng	Dữ liệu	Mức
`users`	username, email, **mật khẩu** (lưu dạng rõ), role, api_key, remember_token	Cao
`credentials`	username, mật khẩu băm MD5, api_key	Cao
`shipments`	mã vận đơn, cảng, trạng thái, **customer_id**	Trung bình
`comments`	nội dung trao đổi của khách hàng theo đơn hàng	Trung bình

Nguồn: `InlaneWeb/db/init.sql`

YC1 QUY TRÌNH NGHIỆP VỤ

Hai nghiệp vụ của người dùng chạm vào dữ liệu cá nhân

① Đăng nhập và quản lý tài khoản

POST /login
username + password

→

Truy vấn bảng users
so khớp tài khoản trong MariaDB

→

Tạo phiên
`$_SESSION['uid']`

→

/account
xem và cập nhật email, thông tin cá nhân

Rủi ro: câu SQL nối chuỗi trực tiếp nên có thể bypass đăng nhập; mật khẩu lưu dạng rõ, lộ ngay khi CSDL bị đọc.

② Tra cứu cảng và theo dõi vận đơn

GET /ports?q=
tìm cảng theo từ khoá

→

/port_detail
thông tin cảng

→

/shipment_detail
trạng thái vận đơn của khách

→

/comment_add
khách trao đổi về đơn hàng

Rủi ro: tham số `q` đi thẳng vào SQL và HTML (SQLi, XSS); xem vận đơn của khách khác.

Các lỗi mã nguồn này do thành viên khác phụ trách. Phần của tôi bảo đảm **mọi bản sửa và mọi thay đổi** chỉ lên production sau khi đã đi qua các cổng kiểm tra đó.

YC1 QUY TRÌNH NGHIỆP VỤ

③ Quy trình phát hành thay đổi (phần tôi phụ trách)

Nhánh cá nhân
namng, haind, manhdx, anhld, mainp
commit theo Conventional Commits

→

PR vào dev
`[namng] fix(ci): ...`
Stage 1 Policy Guard
Stage 2 SonarQube + Semgrep

→

PR dev → main
`release: v1.1.1 - ...`
Stage 1 + Stage 3 smoke test

→

Push lên main
Stage 4 tự động SSH vào VPS, kéo image từ GHCR, chạy production

Ba nguyên tắc bắt buộc

- Không commit hoặc push trực tiếp lên `main` và `dev`
- PR của thành viên chỉ được hướng vào `dev`
- `main` chỉ nhận PR phát hành từ `dev`, tiêu đề theo SemVer

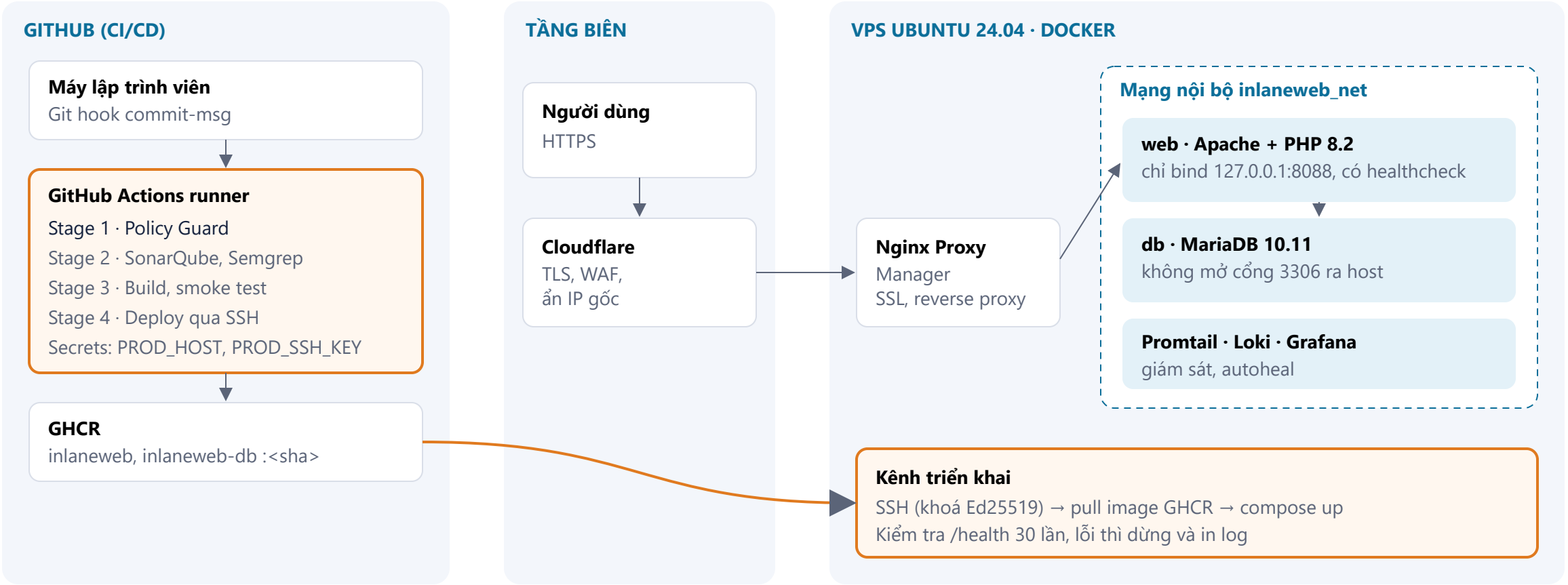
Vì sao đây là vấn đề an toàn

Mỗi lần có code mới trên `main`, Stage 4 **tự động** triển khai lên máy chủ đang giữ dữ liệu khách hàng. Nếu đi tắt được vào `main`, code chưa ai rà soát và chưa qua SAST sẽ chạy thật trên production.

Nguồn: `.github/workflows/devsecops-pipeline.yml`, job `production_deploy_web` chạy khi `github.ref == 'refs/heads/main' && event == 'push'`

YC2 MÔI TRƯỜNG TRIỂN KHAI

Hạ tầng triển khai: GitHub Actions → GHCR → VPS



Nguồn: docs/04, InlaneWeb/docker-compose.prod.yml, devsecops-pipeline.yml

YC2 YÊU CẦU AN TOÀN TỐI THIỂU

Đối chiếu môi trường triển khai với yêu cầu an toàn tối thiểu

Yêu cầu	Hiện trạng (kiểm tra ngày 07/11/2026)	Đánh giá
Cơ sở dữ liệu không lộ ra ngoài	db chỉ nằm trong mạng `inlaneweb_net`, không ánh xạ cổng 3306	Đạt
Web không phơi trực tiếp IP	web bind `127.0.0.1:8088`, đi qua Nginx Proxy Manager và Cloudflare	Đạt
Bí mật triển khai không nằm trong code	Khoá SSH, host nằm ở GitHub Secrets; runner dùng khoá Ed25519 riêng	Đạt
Chuỗi cung ứng CI	Action được ghim theo SHA, `persist-credentials: false`, quyền token tối thiểu	Đạt
Chỉ triển khai bản đã kiểm định	Stage 3 smoke test pass mới đẩy image lên GHCR; Stage 4 kiểm tra `/health` sau khi chạy	Đạt
Mật khẩu CSDL	`MYSQL_ROOT_PASSWORD: password` viết thẳng trong compose	Chưa đạt
Xác thực máy chủ khi SSH	`StrictHostKeyChecking=no`, chưa ghim host key	Cần sửa
Bảo vệ nhánh main / dev	Trước: API trả về `404 Branch not protected`. Đã bật ngày 07/11, API trả về `protected=true`	Đã khắc phục

Dòng cuối là điểm nối sang vấn đề an toàn tôi phụ trách (slide 8–16).

YC3 VẤN ĐỀ AN TOÀN CÁ NHÂN

Kiểm soát phân nhánh lỏng: mã chưa rà soát có thể lên thẳng production

Nếu không có cơ chế ép buộc, bất kỳ ai có quyền ghi vào repo (hoặc một tài khoản bị chiếm) có thể đưa code lên `main` mà **không qua Pull Request, không qua review, không qua SAST**. Stage 4 sẽ tự triển khai code đó lên máy chủ đang giữ dữ liệu khách hàng.

Tiêu chí	Nội dung
Nhóm lỗi	**Triển khai / vận hành** (quy trình tích hợp và phát hành), không phải lỗi trong mã PHP
Pha DevSecOps	**Code / Commit**, thực thi tại **Stage 1** của pipeline, trước mọi stage khác; lớp đầu tiên chạy ngay trên máy lập trình viên
Tài sản bị ảnh hưởng	Tính toàn vẹn của mã chạy trên production và gián tiếp là dữ liệu trong bảng `users`, `shipments`
Kiểu bài toán	**Policy-as-Code**: quy chế phân nhánh viết thành mã Python, kiểm thử tự động, chạy trong CI
Công cụ	`ci/policy_guard.py`, `.githooks/commit-msg`, `tests/test_policy_guard.py`, GitHub API

YC3 CÁCH THỨC PHÁT HIỆN

Phát hiện vi phạm ở 3 lớp, sớm nhất có thể

Lớp 1 · máy trạm

Git hook `commit-msg`

Kích hoạt bằng `git config core.hooksPath .githooks`. Mỗi lần commit, hook gọi `policy_guard.py`; tiêu đề sai chuẩn thì **huỷ commit** ngay trên máy.

Lớp 2 · Stage 1 CI

Policy Guard trên GitHub Actions

- PR vào `main` mà nhánh nguồn khác `dev`: chặn
- Tiêu đề PR vào `dev` / `main` sai mẫu: chặn
- Quét **mọi commit** trong sự kiện push

Stage 1 thất bại thì Stage 2–4 bị bỏ qua.

Lớp 3 · cấu hình repo

Kiểm tra bằng GitHub API

```
gh api repos/wtfwhoyouare/
  PTIT-DevOps-SonarQube/
  branches/main/protection
```

Lần kiểm tra đầu trả về `404 Branch not protected` cho cả `main` và `dev`. Đây là cách tôi phát hiện lỗ hổng còn lại và đã khắc phục (slide 16).

Mô hình hoá luồng hợp lệ: **F(P) = 1** khi (base = dev ∧ head ∉ {main, dev}) ∨ (base = main ∧ head = dev); ngược lại F(P) = 0 và pipeline dừng với `exit 1`.

YC3 CHÍNH SÁCH BỊ VI PHẠM

Vi phạm nguyên tắc bảo vệ dữ liệu ngay từ thiết kế

Nghị định 13/2023/NĐ-CP

- **Điều 26:** biện pháp bảo vệ dữ liệu cá nhân phải được áp dụng **ngay từ khi bắt đầu và trong suốt** quá trình xử lý. Để code chưa kiểm định chạy trên hệ thống xử lý dữ liệu là bỏ trống biện pháp này ở khâu phát hành.
- **Điều 27:** bên xử lý phải có **biện pháp quản lý và kỹ thuật** bảo vệ dữ liệu cá nhân cơ bản. Quy chế phân nhánh chỉ ghi trên giấy mà không ép buộc bằng kỹ thuật thì chưa đáp ứng.

GDPR

- **Article 25 (1):** *data protection by design and by default*: biện pháp kỹ thuật phải có sẵn trong hệ thống, không phụ thuộc vào việc người dùng tự giác.
- **Article 32 (1)(b):** bảo đảm **tính toàn vẹn** liên tục của hệ thống xử lý dữ liệu.
- **Article 32 (1)(d):** có quy trình **kiểm tra, đánh giá định kỳ** hiệu quả của biện pháp kỹ thuật, ở đây là bộ test và pipeline chạy mỗi lần thay đổi.

Hậu quả nếu bị khai thác: backdoor hoặc đoạn code đọc bảng `users` (mật khẩu, email, api_key) có thể lên production trong vài phút, không để lại PR hay review nào để truy vết.

YC3 BÀI TOÁN POLICY-AS-CODE

Từ quy chế viết bằng lời đến luật chạy bằng mã

Quy chế (docs/02 POL-DEVOPS-02)	Luật trong `ci/policy_guard.py`	Kiểm thử
`main` chỉ nhận PR từ `dev`	`if base_branch == "main" and head_branch != "dev": return 1`	có test
PR vào `dev` phải có mã thành viên và loại thay đổi	`PR_DEV_REGEX` = `^\[(namng|haind|manhdx|anhld|mainp)\]\s+(feat|fix|sec|…)…`	có test
PR vào `main` là bản phát hành SemVer	`PR_MAIN_REGEX` = `^release:\s+v[0-9]+\.[0-9]+\.[0-9]+…`	có test
Mọi commit theo Conventional Commits	`COMMIT_REGEX`, áp cho từng commit trong payload push	có test
Không push thẳng lên `main` / `dev`	Branch Protection (`ci/branch-protection.json`, áp qua GitHub API): bắt buộc PR, bắt buộc check Stage 1, áp cả cho admin	đã bật

Có phiên bản: luật nằm trong Git, thay đổi luật cũng phải qua PR

Kiểm thử được: 9 test đơn vị trong `tests/test_policy_guard.py`

Tự động, không né được: chạy ở mọi push và PR vào `dev` / `main`

YC3 CÁCH THỨC XỬ LÝ

Xử lý: chặn sớm, chặn dây chuyền, hướng dẫn sửa

1. Phát hiện
Policy Guard so khớp nhánh và tiêu đề với luật

→

2. Chặn
`exit 1`; Stage 2, 3, 4 phụ thuộc Stage 1 nên không chạy

→

3. Báo lỗi
Annotation `::error` hiện ngay trên trang PR

→

4. Hướng dẫn
In sẵn lệnh sửa: `git commit --amend`, đổi tiêu đề PR

Phụ thuộc giữa các job

```
analyze:                 needs: [branch_policy_guard]
semgrep:                 needs: [branch_policy_guard]
production_smoke_test: needs: [branch_policy_guard,
                                analyze, semgrep]
  if: needs.branch_policy_guard.result == 'success'
production_deploy_*:    needs: [production_smoke_test]
```

Biện pháp tổ chức đi kèm

- Quy chế `docs/02` : ma trận RACI, quy chuẩn PR 2 tầng
- Hook cục bộ được chia sẻ trong repo, cài bằng 1 lệnh
- Lead duyệt PR trước khi merge vào `dev` và `main`
- **Branch Protection** trên `main` và `dev` : GitHub tự từ chối push trực tiếp, PR chỉ merge được khi Stage 1 xanh (slide 16)

YC4 DEMO: ĐOẠN CODE PHÁT HIỆN LỖI

ci/policy_guard.py: kiểm tra luồng Pull Request

```python
def validate_pull_request_event(event_data):
    pr = event_data.get("pull_request", {})
    base_branch = pr.get("base", {}).get("ref", "")
    head_branch = pr.get("head", {}).get("ref", "")
    pr_title = pr.get("title", "").strip()

    # 1. main chỉ nhận PR từ dev
    if base_branch == "main" and head_branch != "dev":
        log_error("Vi phạm luồng phân nhánh", err_msg)
        return 1

    # 2. PR vào dev: [mã_nhánh] loại(phạm_vi): mô tả
    if base_branch == "dev" and not PR_DEV_REGEX.match(pr_title):
        log_error("Sai quy ước đặt tên PR vào dev", err_msg)
        return 1

    # 3. PR vào main: release: vX.Y.Z - mô tả
    if base_branch == "main" and not PR_MAIN_REGEX.match(pr_title):
        log_error("Sai quy ước đặt tên PR vào main", err_msg)
        return 1
    return 0
```

Đầu vào

File JSON của sự kiện mà GitHub đặt ở `$GITHUB_EVENT_PATH`: nhánh nguồn, nhánh đích, tiêu đề PR, danh sách commit.

Đầu ra

`exit 0` cho qua; `exit 1` làm job Stage 1 đỏ, kèm annotation `::error` hiển thị trên GitHub.

Chạy ở đâu

Job `branch_policy_guard` (Stage 1, giới hạn 3 phút) và hook `commit-msg` trên máy trạm. Chỉ dùng thư viện chuẩn Python.

YC4 DEMO: KẾT QUẢ CHẠY TRÊN MÁY

Chạy Policy Guard với 4 sự kiện giả lập và bộ test

```
$ GITHUB_EVENT_NAME=pull_request \
  GITHUB_EVENT_PATH=pr_namng_main.json python ci/policy_guard.py
🔍 [Policy Guard] Kiểm tra Pull Request: [namng] → [main]
⛔ VI PHẠM CHÍNH SÁCH BẢO VỆ NHÁNH MAIN
Nghiêm cấm tạo PR trực tiếp từ nhánh phát triển 'namng' lên 'main'!
exit=1

$ ... pr_dev_badtitle.json   (tiêu đề "fix login")
❌ SAI QUY ƯỚC ĐẶT TÊN PULL REQUEST VÀO NHÁNH DEV
exit=1

$ ... pr_ok.json   (dev → main, "release: v1.1.1 - ...")
✅ Tiêu đề PR vào nhánh 'main' đúng chuẩn phát hành SemVer!
exit=0

$ python ci/policy_guard.py "update code"   # hook commit-msg
❌ Không hợp lệ: ... vi phạm chuẩn Conventional Commits.
exit=1
```

```
$ python -m pytest tests/test_policy_guard.py -v
test_invalid_commit_messages            PASSED
test_pr_to_dev_invalid_title            PASSED
test_pr_to_dev_valid_title              PASSED
test_pr_to_main_invalid_head            PASSED
test_pr_to_main_invalid_release_title   PASSED
test_pr_to_main_valid_release           PASSED
test_push_event_with_invalid_commit     PASSED
test_push_event_with_valid_commits      PASSED
test_valid_commit_messages              PASSED
========== 9 passed in 0.34s ==========
```

Kịch bản	Kỳ vọng	Thực tế
PR namng → main	chặn	chặn
PR vào dev sai tiêu đề	chặn	chặn
PR release dev → main	cho qua	cho qua
Commit "update code"	chặn	chặn

YC4 BẰNG CHỨNG TRÊN GITHUB ACTIONS

Policy Guard đã chặn một lần triển khai production thật

Run **42079784473** · push lên `main` · 05/11/2026 17:58 UTC · release v1.1.0

```
🔍 [Policy Guard] Kiểm tra các commit được push lên ref: refs/heads/main
❌ LỖI VI PHẠM CHÍNH SÁCH ĐẶT TÊN COMMIT (CONVENTIONAL COMMITS)
🚫 Commit [02765ac]: "clean(demo): xoa bo demo_ci.py va giu
   duy nhat demo_live_ci.py de tranh du thua"
##[error]Process completed with exit code 1.

JOBS
X [STAGE 1] Policy Guard                          in 5s
- [STAGE 2: Dev Gate] SonarQube SAST               skipped
- [STAGE 2: Dev Gate] Semgrep Security SAST        skipped
- [STAGE 3: Production Gate] Docker Build          skipped
- [STAGE 4: Production CD] Deploy Web Workload     skipped
- [STAGE 4: Production CD] Deploy Observability    skipped
```

Chuyện gì đã xảy ra

Một commit dùng loại `clean` chưa có trong danh sách hợp lệ lọt vào bản phát hành. Policy Guard quét toàn bộ commit của lần push và dừng pipeline sau 5 giây. **Không có gì được triển khai.**

Xử lý theo đúng quy trình

Sửa luật qua PR #59 `[namng] fix(ci): cap nhat policy guard chap nhan commit type clean`, rồi phát hành lại bằng PR #60 `release: v1.1.1`. Run 42081487042 chạy đủ 4 stage và triển khai thành công.

Nguồn: gh run view 42079784473 · gh run list -R wtfwhoyouare/PTIT-DevOps-SonarQube

PHÁT HIỆN THÊM VÀ KHẮC PHỤC

Điểm hở: push trực tiếp lên main lọt qua Policy Guard

Trước khi khắc phục

```
# Giả lập push thẳng lên main, không qua PR
$ GITHUB_EVENT_NAME=push \
  GITHUB_EVENT_PATH=push_main_direct.json python ci/policy_guard.py
✅ Toàn bộ 1 commit đều tuân thủ chính xác quy chuẩn Conventional
Commits!
exit=0   ← commit "feat(auth): them tai khoan quan tri an" được cho qua

$ gh api repos/.../branches/main/protection
{"message":"Branch not protected","status":"404"}   # dev: tương tự
```

Với sự kiện push, Policy Guard chỉ kiểm tra **tên commit**, không biết commit có đến từ PR hay không. Push đúng tên lên `main` sẽ đi thẳng tới Stage 4.

Đề xuất thêm (chưa đưa vào repo): luật trong Policy Guard yêu cầu push lên `main` / `dev` phải là merge commit của PR, làm lớp kiểm tra thứ hai.

Đã khắc phục: Branch Protection as Code (07/11/2026)

```
# ci/branch-protection.json, áp cho main và dev
{ "required_status_checks": { "checks": [
    { "context": "[STAGE 1] Policy Guard — ..." } ] },
  "required_pull_request_reviews": { ... },
  "enforce_admins": true,
  "allow_force_pushes": false,
  "allow_deletions": false }

$ gh api -X PUT repos/.../branches/main/protection \
    --input ci/branch-protection.json
$ gh api repos/.../branches/main --jq .protected
main: protected=true
dev:  protected=true
```

- Mọi thay đổi vào `main` / `dev` **bắt buộc qua PR**
- PR chỉ merge được khi **Stage 1 Policy Guard xanh**
- Áp dụng cả cho admin; cấm force-push và xoá nhánh

KẾT LUẬN

Tóm tắt phần việc cá nhân

4
stage pipeline do tôi thiết kế; Stage 1 là cổng mà mọi thay đổi phải qua đầu tiên

9/9
test của Policy Guard pass; 4/4 kịch bản demo cho kết quả đúng kỳ vọng

1
lần triển khai production thật bị Policy Guard chặn (run 42079784473)

Yêu cầu	Kết quả
Vấn đề an toàn	Kiểm soát phân nhánh lỏng, nhóm lỗi **triển khai / vận hành**, pha **Code / Commit** (Stage 1)
Phát hiện	Git hook, Policy Guard ở Stage 1, kiểm tra cấu hình repo bằng GitHub API
Xử lý	Chặn bằng `exit 1`, ngắt Stage 2–4, hướng dẫn sửa; bật Branch Protection cho `main` và `dev` (07/11)
Chính sách	NĐ 13/2023 Điều 26, 27; GDPR Article 25(1), 32(1)(b)(d)
Policy-as-Code	Quy chế `docs/02` viết thành `ci/policy_guard.py`, có phiên bản, có test, chạy tự động